

Malitsky–Pock’s Linesearch, as a Linesearch on LADMM’s Penalty

Optimal Transport / Schrödinger Bridge Project
Companion to `adaptive_pdhg_goldstein.tex`

Abstract

Malitsky & Pock, *A first-order primal-dual algorithm with linesearch*, SIAM J. Optim. 28(1) 411–432, 2018 (arXiv:1608.08883), Algorithm 1, applied to this project’s *dual* saddle point – their $f = f_1^*$, their $g = f_2^*$, their $K = -A^\top$ – is **exactly `ladmm.py`’s own iteration**, in its original (r, x, y, δ) cycle, with no reordering and no lag. The reason is a single coincidence that the companion document had already found and read as an obstruction: their extrapolation coefficient $\theta_k = \tau_k / \tau_{k-1}$ is *identical* to the ratio $\rho_k = \gamma_k / \gamma_{k-1}$ that LADMM’s unrotated cycle produces on its own. What blocked the match with Goldstein–Li–Yuan’s fixed $(2, -1)$ is precisely what makes this one work. Their τ_k *is* the LADMM penalty γ_k , and their fixed ratio β ties the proximal step to it by $\tau_k^{\text{ladmm}} = 1/(\beta\gamma_k)$. So the linesearch is a linesearch on γ , holding $\gamma\tau$ fixed – orthogonal, in $(\log \gamma, \log \tau)$, to Goldstein–Li–Yuan’s rule, which holds γ/τ fixed. Part VI reports what it actually buys: complete insensitivity to the initial step, and no speedup over a well-tuned one.

Part I – their Algorithm 1, renamed to avoid this project’s letters

Same discipline as the companion document: keep their symbol unless it already means something here, otherwise pick a fresh one – and, in particular, do *not* give their step this project’s name for a step. Their τ_k is written λ_k throughout Parts I and II, as a symbol with no meaning here yet. Part III then *derives* $\lambda_k = \gamma_k$; writing γ before that point would assume the result the document exists to prove.

Their symbol	What it is	Written here as
x^k (updated by prox_g)	clashes with Part I’s x	v^k
y^k (updated by prox_{f^*})	clashes with Part I’s y	u^k
τ_k (their step)	clashes with Part I’s τ	λ_k
θ_k (extrapolation)	clashes with the companion’s θ	ρ_k
β ’s partner $\sigma_k = \beta\tau_k$	clashes with σ	$\beta\lambda_k$
δ (linesearch tolerance)	clashes with the multiplier δ	κ
β (step ratio σ/τ)	no clash	β
μ (backtracking factor)	no clash	μ
K (operator)	$= -A^\top$	K
$L = \ K\ $	no clash	$\ A\ $

Their saddle point is $\min_x \max_y g(x) + \langle Kx, y \rangle - f^*(y)$, and Algorithm 1 (PDAL) reads, with x^0, y^1 given, $\lambda_0 > 0$, $\mu, \kappa \in (0, 1)$, $\beta > 0$, $\rho_0 = 1$:

$$1. \quad v^k = \text{prox}_{\lambda_{k-1}g} (v^{k-1} - \lambda_{k-1}K^*u^k) \quad (1)$$

2. choose $\lambda_k \in [\lambda_{k-1}, \lambda_{k-1}\sqrt{1 + \rho_{k-1}}]$, then linesearch:

$$2a. \quad \rho_k = \frac{\lambda_k}{\lambda_{k-1}}, \quad \bar{v}^k = v^k + \rho_k(v^k - v^{k-1}), \quad u^{k+1} = \text{prox}_{\beta\lambda_k f^*} (u^k + \beta\lambda_k K\bar{v}^k) \quad (2)$$

$$2b. \quad \text{break if } \sqrt{\beta}\lambda_k \|K^*u^{k+1} - K^*u^k\| \leq \kappa \|u^{k+1} - u^k\|; \text{ else } \lambda_k \leftarrow \mu\lambda_k \text{ and repeat 2a.} \quad (3)$$

Note $\bar{v}^k = (1 + \rho_k)v^k - \rho_kv^{k-1}$: a *step-ratio-dependent* extrapolation, not a fixed one. That is the whole point of what follows.

Part II – which saddle point

Take their $f := f_1^*$ and their $g := f_2^*$, so their $f^* = f_1$. Their saddle point becomes

$$\min_v \max_u f_2^*(v) + \langle Kv, u \rangle - f_1(u), \quad K := -A^\top, \quad (4)$$

which is *the same saddle point* the companion document's Part II derives as the Fenchel dual of $\min_x f_1(x) + f_2(Ax)$ – so both proxes are ones this project already computes: prox_{f_1} is the FP-feasibility projection (an indicator, hence step-independent) and $\text{prox}_{tf_2^*}$ comes from $\text{prox}_{f_2/t}$ by Moreau. With $K^* = -A$, (1) and (2) read

$$v^k = \text{prox}_{\lambda_{k-1}f_2^*} (v^{k-1} + \lambda_{k-1}Au^k) \quad (5)$$

$$u^{k+1} = \text{prox}_{\beta\lambda_k f_1} (u^k - \beta\lambda_k A^\top \bar{v}^k) \quad (6)$$

Part III – it is `ladmm.py`, unrotated and unlagged

Write `ladmm.py`'s own cycle (its (r, x, y, δ) order, $\alpha = 1$, d folded into the affine A), with per-iteration γ_k and τ_k :

$$D^k = Ax^k - y^k$$

$$x^{k+1} = \text{prox}_{(1/\tau_k)f_1} \left(x^k - \frac{1}{\tau_k} A^\top (\gamma_k D^k - \delta^k) \right) \quad (7)$$

$$y^{k+1} = \text{prox}_{f_2/\gamma_k} (Ax^{k+1} - \delta^k/\gamma_k)$$

$$\delta^{k+1} = \delta^k - \gamma_k (Ax^{k+1} - y^{k+1}) \quad (8)$$

Set $u^k := x^k$ and $v^k := -\delta^k$.

The y/δ pair is (5). By the same Moreau step the companion document verifies,

$$\text{prox}_{\gamma_k f_2^*} (\gamma_k Ax^{k+1} - \delta^k) = \gamma_k (Ax^{k+1} - y^{k+1}) - \delta^k = -\delta^{k+1},$$

i.e. $v^{k+1} = \text{prox}_{\gamma_k f_2^*} (v^k + \gamma_k Au^{k+1})$, which is (5) shifted by one index – *provided* their step is this penalty. ✓ That is the first identification:

$\lambda_k = \gamma_k$

i.e. Parts I–II’s placeholder λ_k (their τ_k) is LADMM’s penalty. From here on γ_k is written for both, which is now a theorem rather than a naming convention.

The x -update’s bracket is their $\bar{v}^k$, for free. This is the step that needs no lag and no reordering. The δ -update (8) at $k - 1$ gives $\gamma_{k-1}D^k = \delta^{k-1} - \delta^k$, so $\gamma_k D^k = \rho_k(\delta^{k-1} - \delta^k)$ with $\rho_k = \gamma_k/\gamma_{k-1}$, and (7)’s bracket is

$$\gamma_k D^k - \delta^k = \rho_k \delta^{k-1} - (1 + \rho_k) \delta^k \stackrel{\delta=-v}{=} (1 + \rho_k) v^k - \rho_k v^{k-1} = \bar{v}^k.$$

`ladmm.py`’s existing gradient bracket *is* Malitsky–Pock’s extrapolated iterate, with ρ_k their θ_k . No auxiliary variable, no stored δ^{k-1} , no shifted index: the extrapolation is implicit in the bracket the code already forms. ✓ Matching the prox step in (7) against (6) gives $1/\tau_k = \beta\gamma_k$, so

$$\boxed{u^k = x^k, \quad v^k = -\delta^k, \quad K = -A^\top, \quad \lambda_k = \gamma_k, \quad \rho_k = \frac{\gamma_k}{\gamma_{k-1}}, \quad \tau_k = \frac{1}{\beta\gamma_k}}$$

Why this worked where the fixed $(2, -1)$ did not. The companion document’s Part VI computes exactly the same bracket and gets $(1 + \rho_k, -\rho_k)$, and treats it as the obstruction to matching Goldstein–Li–Yuan’s literal $(2, -1)$ – an obstruction that forced either a reordering of the cycle or a lag. Malitsky–Pock’s extrapolation coefficient is *defined* to be the step ratio, so the same ρ_k that broke the other match is what makes this one exact. Nothing had to move.

Part IV – the linesearch, in LADMM’s own quantities

Substituting $K^* = -A$ and $u = x$ into (3), and noting the affine part of A cancels in the difference:

$$\boxed{\sqrt{\beta}\gamma_k \|A(x^{k+1} - x^k)\| \leq \kappa \|x^{k+1} - x^k\|}$$

so one iteration of the scheme is: given $x^k, y^k, \delta^k, \gamma_{k-1}, \rho_{k-1}$,

- (1) $D^k = Ax^k - y^k$ (against the *stored* y^k – unrotated).
- (2) trial $\gamma_k \in [\gamma_{k-1}, \gamma_{k-1}\sqrt{1 + \rho_{k-1}}]$; then repeat: $\rho_k = \gamma_k/\gamma_{k-1}$, $\tau_k = 1/(\beta\gamma_k)$, $x^{k+1} = \text{prox}_{(1/\tau_k)f_1}(x^k - \frac{1}{\tau_k}A^\top(\gamma_k D^k - \delta^k))$; accept if the boxed test holds, else $\gamma_k \leftarrow \mu\gamma_k$.
- (3) y^{k+1}, δ^{k+1} from (8) with the *same* γ_k .

Initialization, spelled out rather than left implicit. Their Algorithm 1 runs step 1 – the y/δ update – *before* the first linesearch, so that the first x -update already has a genuine (δ^k, δ^{k-1}) pair to extrapolate from. A LADMM loop started straight from $\delta^0 = 0$ does not: its first bracket is $\gamma_0 D^0 - \delta^0 = v^1$, short of $(1 + \rho_1)v^1$ by the entire extrapolation factor, and every later iterate inherits the discrepancy. `ladmm_linesearch.py` therefore primes with one y/δ update before the loop, and ignores the y^0 it is passed (it manufactures y^1 itself, as their step 1 does). Checked both ways: without priming the solver departs from a direct transcription of Algorithm 1 by the fourth iteration; with it, the step sequences γ_k agree to *exactly* zero over 40 iterations (with backtracking active throughout) and the final iterates to 1.4×10^{-15} .

On *this* problem the priming happens to change nothing, for the same structural reason $\mathbf{dz}^0$ is machine zero: the initial guess carries $m = 0$, so Ax^0 has zero momentum, the KE prox of a

zero-momentum state is the identity, $y^1 = Ax^0$ and $D^0 = 0$ to 9×10^{-16} – so $\delta^1 = \delta^0$ and the two initializations coincide. Every number in Part VI is bit-identical before and after the fix. It would *not* be a no-op for a warm start, or any initial guess with nonzero momentum.

Two implementation notes, both from their Remark 1. First, only ρ_k varies inside the linesearch, so $A^\top(\gamma_k D^k - \delta^k) = -(1 + \rho_k)A^\top \delta^k + \rho_k A^\top \delta^{k-1}$ is a combination of two adjoints cached once per iteration – no $A^\top$ per trial. Second, and less comfortably: their Remark 1 advises choosing as the linesearched variable the one whose prox is *cheaper*. Under the mapping specified here, the linesearched prox is prox_{f_1} – the FP projection, this solver’s expensive operation. Every rejected trial costs one projection. The honest work metric is therefore the projection count, not the iteration count, and Part VI reports both.

Part V – how this relates to the other adaptive rules

Because β is fixed, $\tau_k = 1/(\beta\gamma_k)$ means

$$\gamma_k \tau_k = \frac{1}{\beta} = \text{const}, \quad \text{so} \quad \frac{\gamma_k}{\tau_k} = \beta \gamma_k^2 \text{ varies.}$$

Set against Goldstein–Li–Yuan’s Algorithm 1 rule (companion document, Part V), which scales γ and τ by the *same* factor and so holds γ/τ invariant while moving $\gamma\tau$:

	holds fixed	moves
Goldstein–Li–Yuan residual balancing	γ/τ	$\gamma\tau$
Malitsky–Pock linesearch (this note)	$\gamma\tau$	γ/τ

In $(\log \gamma, \log \tau)$ the two move along orthogonal directions, and together they span the whole step-size plane. That matters because the companion work found the remaining headroom on this problem to be in the *ratio*: the stability bound $\tau > \gamma\|A\|^2$ is $\beta\gamma^2 < 1/\|A\|^2$ here, and the boxed linesearch test is its *directional* version – $\|A\delta x\|/\|\delta x\|$ in place of $\|A\|$ – so it can accept steps the worst-case bound forbids whenever the increment is not aligned with A ’s leading singular vector. It reaches, without a backtracking safeguard on the ratio, the direction Goldstein–Li–Yuan’s rule structurally cannot move in.

Note also what β is: since $\gamma/\tau = \beta\gamma^2$, choosing β is choosing the operating ratio. The linesearch removes the need to tune γ , but β takes its place as the one free parameter.

Part VI – what it actually buys

All runs: `sb_gaussian_nonuniform`’s Gaussian SB, clustered grid, ETD, $n_t = n_x = 32$, $\varepsilon = 10^{-4}$, 4000 iterations, against fixed-step LADMM at the project default. “projections” counts linesearch trials, since each repeats prox_{f_1} .

The equivalence, checked numerically. A direct transcription of Algorithm 1 in their own letters and the LADMM form of Part III, run from the same start with the same linesearch: iterates agree to 1.8×10^{-15} , and the step sequences γ_k agree *exactly* for the first 55 of 60 iterations. They part only once $\|x^{k+1} - x^k\|$ reaches $\sim 10^{-15}$, where the linesearch test compares two rounding-noise quantities – the same benign post-convergence effect the companion document records for the other rules.

Insensitivity to the initial step: yes, decisively.

γ_0	fixed-step LADMM		with linesearch	
	iters to $D < 10^{-5}$	mean L^2	iters to $D < 10^{-5}$	γ ends at
0.01	never	1.95×10^{-2}	3553	1.284
0.1	never	1.97×10^{-2}	3494	1.311
1	3330	2.14×10^{-2}	3202	1.339
10	1472	2.78×10^{-1}	3226	1.457
100	981	8.97×10^{-1}	3226	1.188

From γ_0 spanning four orders of magnitude the linesearch lands in the same place every time (γ settling in 1.19–1.46, mean L^2 within 1.5% across all five). Fixed-step LADMM does not: below $\gamma_0 = 1$ it never reaches the residual threshold at this budget, and above it the residual falls *faster* while the answer gets steadily worse – $\gamma_0 = 100$ reaches $D < 10^{-5}$ in 981 iterations with a mean L^2 error of 0.90 against the reference’s 0.02. That is the familiar warning that a small residual is not a correct answer, in its sharpest form so far.

Faster convergence: no, not on this problem. Against a *well-tuned* fixed step the linesearch does not win:

	iters	projs	iters to $D < 10^{-5}$	projs to $D < 10^{-5}$
LADMM, $\gamma = 1, \tau = 1.01$	4000	4000	3330	3330
linesearch, $\beta = 1$, growth = 1	4000	7905	3648	7209
linesearch, $\beta = 1$, growth = 0.2	4000	4896	3698	4526
linesearch, $\beta = 0.5$, growth = 0.2	4000	4895	3326	4070

At best it matches the tuned run’s iteration count (3326 vs 3330) while spending 22% more projections. The “growth” column is the position in their step-2 interval: at the largest trial ($= 1$, their first option) the step always over-reaches once γ has settled and almost every iteration wastes a rejected trial – 1.98 trials/iteration measured – which their Step 2 explicitly lets us dial back, to 1.22 at growth = 0.2.

Verdict. The linesearch buys robustness, not speed: it removes γ from the list of things that must be tuned, and it is the only mechanism examined in this project that moves γ/τ at all. It does not accelerate a run whose γ was already well chosen, and it costs one FP projection per rejected trial. Whether it pays therefore depends on whether a good γ is known in advance – which, for the grid/ ε /scheme sweeps this project runs, it often is not.

Not claimed. Only $\varepsilon = 10^{-4}$, one grid, one scheme, one resolution, and a fixed 4000-iteration budget were tested; β was swept only over $\{0.25, \dots, 8\}$. The accelerated variants (their Algorithms 2 and 3, for strongly convex g or f^*) are not implemented, and neither is their Algorithm 4.